Java String and immutability

Contents

What a String is, why it is a class, what immutability and mutability mean, the String Constant Pool, and the classic traps like == v/s equals() and storing passwords. Definitions stay tight; the answer does the teaching.

1. What is a String in Java?

A String is an immutable object that represents a sequence of characters. It is an instance of the String class in the `java.lang` package.

```
String name = "Rohit";
String city = new String("Chennai");
```

Both create String objects.

Why a String is an object: a common beginner assumption is that a string is a primitive. It is not.

```
String name = "Java";
```

Here:

- `String` is the class.
- `name` is the reference variable.
- `"Java"` is the String object.

That is why we can call methods on it:

```
System.out.println(name.length());
System.out.println(name.toUpperCase());
System.out.println(name.substring(1));
```

Strings are immutable: once created, a string cannot be changed.

```
String s = "Hello";
s.concat(" World");
System.out.println(s);
```

Output:

```
Hello
```

Nothing changed, because `concat()` returns a new string rather than modifying `s`. To keep the result, we reassign it:

```
String s = "Hello";
s = s.concat(" World");
System.out.println(s);
```

Output:

```
Hello World
```

So `concat()` builds a brand-new `"Hello World"` object and leaves the original `"Hello"` untouched. The variable `s` simply points at the new one.

2. Why is String a class and not a primitive type?

Because a String holds complex data and needs rich behaviour that a primitive cannot give. Being a class lets Java attach methods, immutability, object-oriented features, and internal optimisations like the String Pool.

Why not a primitive: primitives are built to store simple values, like `int`, `char`, `boolean`, and `double`. They keep a raw value, have a fixed size, and carry no methods.

```
int x = 10;
x.length(); // invalid: primitives have no methods
```

A String needs far more than just storing characters, which is why it is a class.

It provides methods:

```java
String name = "Rohit";
name.length();
name.substring(2);
name.toUpperCase();
name.contains("hit");
```

If String were a primitive, none of these would exist.

It supports object-oriented programming: since String is a class, it inherits from `Object`.

```java
String str = "Java";
System.out.println(str.hashCode());
System.out.println(str.equals("Java"));
System.out.println(str.toString());
```

Those methods come from the object model.

3. What does immutability mean?

Immutability means that once an object is created, its state cannot be changed. If we want a different value, a new object is created rather than the existing one being modified.

For String this is exactly what question 1 showed: `s.concat(" World")` never touches the original `"Hello"`, it returns a new `"Hello World"` object, and unless we reassign `s`, the original value stays put. The upshot is that an immutable object is safe to share freely, because nothing can alter it after creation.

4. What does mutability mean?

Mutability is the opposite: an object's state can be changed after it is created. Instead of making a new object, the existing one is modified in place. `StringBuilder` is the mutable counterpart to String.

```java
StringBuilder sb = new StringBuilder("Hello");
sb.append(" World");
System.out.println(sb);
```

Output:

```
Hello World
```

The same object is modified, no new one is created.

Mutable v/s immutable: the same operation, side by side.

Mutable (`StringBuilder`):

```
StringBuilder sb = new StringBuilder("Java");
sb.append(" 21");
System.out.println(sb); // Java 21
```

The original object changes.

Immutable (`String`):

```
String str = "Java";
str.concat(" 21");
System.out.println(str); // Java
```

`concat()` does create a new object, but since its reference is never assigned, the original string is left unchanged.

5. What is the String Constant Pool (SCP), and why was it introduced?

The String Constant Pool is a special area in the JVM heap that stores unique string literals. If a literal already exists in the pool, Java hands back the existing object instead of making a new one.

Why it was introduced:

- Cut memory use by avoiding duplicate string objects.
- Improve performance by reusing existing instances.
- Lean on String immutability, which makes it safe for many references to share one object.

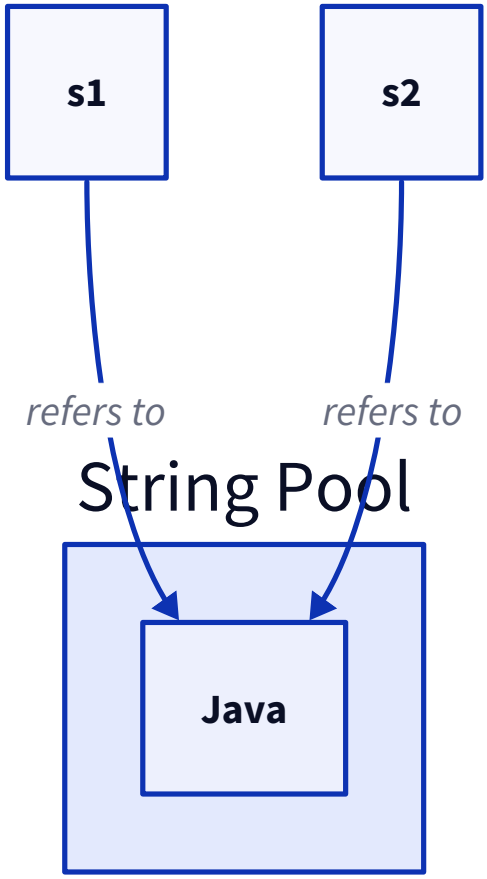
How it works: when we create a string literal:

```
String s1 = "Java";
```

the JVM checks the pool. If `"Java"` is already there, it returns the existing object; otherwise it creates one and stores it.

```
String s1 = "Java";
String s2 = "Java";
System.out.println(s1 == s2); // true
```

Both `s1` and `s2` end up pointing at the same object in the pool:



6. Where is the String Pool stored?

In modern Java (7 and later) the String Pool lives in the heap. In Java 6 and earlier it lived in PermGen (the permanent generation). It was moved to the heap because PermGen had a fixed, limited size, so pools with many interned strings could exhaust it and throw `OutOfMemoryError: PermGen space`. On the heap the pool is covered by normal garbage collection and is not boxed in by that fixed limit.

7. Difference between == and equals()

`==` compares references, whether two variables point to the same object (for primitives, it compares the values). `equals()` compares the contents, the logical equality of two objects, and its behaviour depends on how the class implements it.

Feature	`==`	`equals()`
Compares	Reference for objects, value for primitives	Object content (logical equality)
Works with primitives	Yes	No
Works with objects	Yes	Yes
Default implementation	N/A	Inherited from `Object`, which compares references
Can be overridden	No	Yes

The catch worth remembering: unless a class overrides `equals()`, the inherited `Object` version just does a `==` reference check. String overrides it to compare characters, which is why `equals()` on two strings compares their text.

8. Why are passwords not recommended to be stored as String?

Because String is immutable. Once created, its contents stay in memory until the object is garbage collected, and we cannot explicitly wipe the password out. Java recommends a `char[]` instead, so the password can be overwritten the moment we are done with it.

Why String is a problem:

```
String password = "mySecret123";
```

It is immutable, so we cannot modify or clear its contents, and it lingers in memory until garbage collection, whose timing is unpredictable. Even after:

```
password = null;
```

the actual `"mySecret123"` object can still sit in memory until the GC happens to run.

Why char[] is better: we can overwrite it in place as soon as we are finished.

```
char[] password = {'m', 'y', 'S', 'e', 'c', 'r', 'e', 't'};

// after using it
Arrays.fill(password, '\0');
```

The array now holds `['\0', '\0', '\0', '\0', '\0', '\0', '\0', '\0']`, so the sensitive data is gone immediately rather than waiting on the garbage collector.

9. If strings are immutable, how does concat() appear to modify them?

It never actually does. `concat()` builds a new String holding the joined result and returns it; the original is left untouched. The only reason it can look like a modification is when the return value is ignored, so `s.concat(" World")` on its own changes nothing visible, which is exactly the case shown in question 1. The result sticks only if we assign it back with `s = s.concat(" World")`, and even then `s` is pointing at a brand-new object, not a mutated one. So there is no modification happening anywhere, just a fresh object and a reassigned reference.